



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 2

Model bahasa, dari sisi pembangun agen

Bukan cara kerja Transformer — itu ada di Bab 15. Ini bagian yang mengubah keputusan: token sebagai satuan tagihan, konteks yang dibayar ulang, dan panggilan alat sebagai perilaku terlatih.

Duration 3 jam (2 sesi)

Instructor Hendri Karisma

Source Grootendorst & Alammari, An Illustrated Guide to AI Agents — bab 2

Session Objectives

By the end of this session, participants can:

- 1 **Menghitung tagihan sebuah percakapan** dari panjang giliran dan jumlah giliran, dan menjelaskan kenapa ia tumbuh kuadrat.
- 2 **Menyebutkan apa yang dipertahankan singgahan prompt** dan satu kebiasaan yang mematikannya tanpa peringatan.
- 3 **Menjelaskan bahwa model tidak punya ingatan antar giliran**, dan menunjukkan di mana keadaan percakapan sebenarnya disimpan.
- 4 **Menerangkan panggilan alat sebagai keluaran terstruktur** yang belum dijalankan, dan menunjuk baris tempat ia dijalankan.
- 5 **Membedakan tiga tahap pelatihan** dan menyebutkan tahap mana yang membuat sebuah model bisa dipakai sebagai agen.
- 6 **Menyebutkan tiga sifat model** yang harus diukur sebelum memilihnya untuk sebuah agen — dan mana yang tidak bisa dibaca dari papan skor.

SITE

[Course home](#)

BOOK

[Maarten Grootendorst & Jay Alammar, An Illustrated Guide to AI Agents \(O'Reilly Media, Early Release\)](#)

Yang sengaja TIDAK diulang di sini

Sudah dibahas di tempat lain

- ♦ Cara perhatian diri menghitung — Bab 15, dengan angkanya
- ♦ Penyandian posisi, encoder lawan decoder — Bab 15
- ♦ Pelatihan awal dan penyetelan halus — Bab 15–16

Yang dibahas di sini

- ♦ Token sebagai satuan tagihan, waktu, dan batas
- ♦ Kenapa percakapan panjang mahal secara kuadrat
- ♦ Panggilan alat sebagai keluaran terstruktur
- ♦ Sifat model mana yang menentukan keberhasilan agen

Aturannya sederhana: yang masuk ke sini hanya yang **mengubah keputusan** saat membangun agen.

01

Token: satuan tagihan, waktu, dan batas

Tiga hal yang berbeda, semuanya diukur dengan satuan yang sama.

Model tidak melihat kata, dan tidak melihat huruf

Kenapa ini penting bagi pembangun agen, bukan bagi ahli bahasa: **semua yang Anda bayar, semua yang Anda tunggu, dan semua batas yang Anda tabrak dihitung dalam token** — bukan dalam karakter, kalimat, atau permintaan.

Taksiran kasar yang cukup untuk perencanaan: **satu token $\approx$ 4 karakter** untuk teks Inggris, dan lebih boros untuk bahasa Indonesia serta untuk JSON — dan hasil alat Anda hampir selalu JSON.

Masukan dan keluaran tidak berharga sama, dan tidak berperilaku sama

Token masukan — perintah sistem, skema alat, seluruh riwayat, semua hasil alat. Dibaca sekaligus, sejajar, jadi cepat.

Di sebuah agen, inilah yang membengkak. Bukan jawabannya.

Token keluaran — yang ditulis model. Dihasilkan satu per satu, tiap token menunggu token sebelumnya, jadi **lambat**.

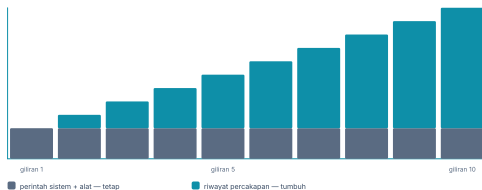
Akibat praktisnya: **agen adalah beban masukan**.

Lebih mahal per token, tapi di sebuah agen jumlahnya jauh lebih sedikit: panggilan alat itu pendek.

Mengoptimalkan panjang jawaban hampir tidak mengubah apa pun; memendekkan yang masuk mengubah semuanya.

Percakapannya tumbuh lurus, tagihannya tidak

Isi konteks pada tiap giliran



percakapan akhir

3 950 token

yang ditagihkan

23 750 token

6.0× lipat

Percakapannya tumbuh lurus; tagihannya tumbuh kuadrat. Tiap giliran membayar ulang semua giliran sebelumnya.

Sepuluh giliran, dihitung dari perintah sistem 800 token dan 350 token per giliran. Langkah: giliran pertama, lima pertama, lalu sepuluh.

Percakapan yang berakhir pada 3 950 token menagihkan **23 750** token masukan — enam kali lipat. Dan ini bukan sifat khusus agen; ini sifat percakapan mana pun. Yang khusus pada agen adalah **giliran per tugasnya banyak**, jadi ia berada jauh di kanan grafik itu.

Dua obatnya, dan yang kedua sering dirusak sendiri

Kurangi gilirannya

Satu langkah yang dihapus menghemat lebih banyak daripada seribu token yang dipangkas, sebab langkah itu dibayar ulang oleh **semua** langkah sesudahnya.

Singgahkan bagian yang tetap

Perintah sistem, skema alat, dan teks kebijakan tidak berubah antar giliran. Penyedia menagihnya jauh lebih murah kalau awalnya **sama persis**.

Satu stempel waktu di dalam perintah sistem membuat awalnya berbeda tiap kali, dan **singghannya mati tanpa satu pun pesan galat**. Periksa `cache_read_input_tokens` bukan nol — jangan percaya bahwa ia menyala.

Jendela konteks adalah dinding, bukan saran

Yang paling awal biasanya **perintah sistem dan batasan Anda**. Jadi kegagalannya berbentuk: agen bekerja normal selama sepuluh giliran, lalu pada giliran ke dua puluh mulai melanggar aturan yang sudah tidak ada lagi di konteksnya.

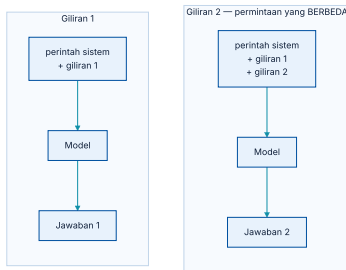
Karena itu memangkas riwayat harus jadi **keputusan yang Anda tulis**, bukan akibat sampingan. Bab 4 membahas apa yang disimpan dan apa yang dibuang.

02

Dari model bahasa ke mesin agen

Tiga hal yang harus ada sebelum sebuah model bisa dipakai dalam gelung.

Model tidak punya ingatan — riwayatnya yang punya



Dua giliran adalah dua permintaan yang sama sekali terpisah.

Antara dua giliran, model **tidak menyimpan apa pun**. Yang membuatnya tampak mengingat adalah kode Anda mengirim ulang seluruh percakapan tiap kali. Keadaan percakapan ada di **daftar pesan milik Anda**, dan itu kabar baik: keadaan yang Anda pegang adalah keadaan yang bisa Anda periksa, potong, dan simpan.

Perintah sistem: kontrak, bukan mantra

Yang pantas ditaruh di sana

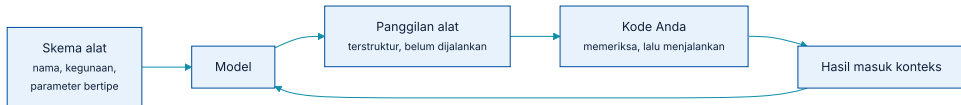
- ♦ Peran dan pembacanya
- ♦ Format jawaban yang diharapkan
- ♦ Kapan harus menyerah dan bertanya
- ♦ Kebijakan yang harus dikutip, bukan diingat

Yang tidak boleh hanya di sana

- ♦ "Jangan pernah menghapus data" → jangan beri alat hapus
- ♦ "Jangan bocorkan data pribadi" → jangan beri alat yang bisa membacanya
- ♦ "Jangan menyetujui" → jangan sediakan alat menyetujui

Uji sederhana: kalau sebuah kalimat di perintah sistem hilang, apakah sistemnya jadi tidak aman? Kalau ya, **kalimat itu berada di tempat yang salah.**

Panggilan alat adalah keluaran terstruktur, bukan eksekusi



Model menghasilkan niat; kode memutuskan apakah niat itu dijalankan.

Model diberi **skema** tiap alat — namanya, kegunaannya, parameternya beserta tipe. Yang ia keluarkan hanyalah sebuah objek: nama alat dan argumen. Objek itu belum melakukan apa-apa.

Ini fakta arsitektural terpenting dalam seluruh modul, dan ia muncul lagi di tiap bab: **model menuliskan niat, kode Anda yang mengeksekusi**. Semua batas izin yang Anda punya berada di celah itu.

Deskripsi alat adalah antarmuka, dan ditulis untuk pembaca yang aneh

DITULIS BEGINI	AKIBATNYA	PERBAIKANNYA
<code>cari(q)</code> — mencari data	Dipanggil untuk apa saja, termasuk yang bukan urusannya	Sebut apa yang dicari dan kapan memakainya
Dua alat dengan deskripsi mirip	Dipilih acak antara keduanya	Gabungkan, atau bedakan dengan tegas
Parameter tanpa tipe atau contoh	Argumen ngawur, galat di dalam alat	Tipe ketat + satu contoh nilai
Deskripsi menyebut cara kerjanya	Tidak membantu memilih	Sebut hasilnya , bukan implementasinya

Suhu: satu angka yang diam-diam menentukan keterulangan

Rendah (0–0,3)

Nyaris sama tiap kali. Yang Anda mau untuk **memilih alat** dan mengeluarkan JSON.

Sedang (0,5–0,7)

Untuk menyusun penjelasan yang enak dibaca, sesudah keputusannya diambil.

Tinggi (> 1)

Untuk mencari ragam gagasan. Hampir tidak pernah untuk agen yang menyentuh sistem nyata.

Dan satu jebakan pengujian: pada suhu tinggi, **kumpulan uji yang lulus hari ini bisa gagal besok tanpa satu baris pun berubah**. Kalau hasil uji Anda goyah, periksa angka ini sebelum menyalahkan agennya.

Hasil alat adalah JSON, dan JSON itu boros

Satu tabel 20 baris dengan 8 kolom bisa menghabiskan beberapa ribu token — dan karena ia masuk riwayat, ia **dibayar ulang di tiap giliran sesudahnya**. Ini penyebab pembengkakan biaya yang paling sering tidak terlihat, sebab yang dilihat orang adalah panjang jawabannya.

Kembalikan yang dipakai

Alat yang mengembalikan seluruh baris ketika agen hanya butuh satu angka membayar seribu token untuk satu.

Ringkas di sisi alat

Peringkasan yang dilakukan **kode** gratis dan pasti; yang dilakukan model berbayar dan bisa salah.

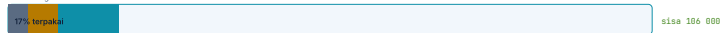
Rujuk, jangan salin

Simpan hasil besar di luar, kembalikan pengenalnya. Bab 4 menyebut ini memori kerja.

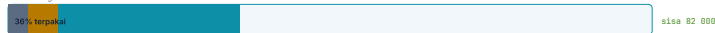
Jendela konteks itu anggaran, dan sebagian sudah habis sebelum mulai

jendela 128k token, dibelanjakan

setelah 5 giliran



setelah 15 giliran



setelah 30 giliran

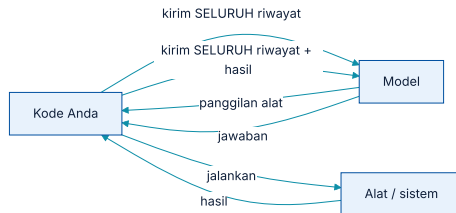


10 000 token sudah terpakai sebelum giliran pertama — dan tiap alat yang ditambahkan memotong lagi.

Jendela 128k, dibelanjakan: bagian tetap, lalu hasil alat yang menumpuk tiap giliran. Langkahi 5, 15, dan 30 giliran.

Sepuluh ribu token sudah terpakai **sebelum giliran pertama**, dan tiap alat baru memotong lagi. Itu sebabnya “tambahkan saja satu alat lagi” bukan keputusan gratis: ia memperkecil ruang kerja tiap giliran sesudahnya.

Satu giliran agen adalah dua perjalanan penuh



Memanggil satu alat berarti mengirim seluruh riwayat dua kali.

Model memutuskan alat mana yang dipanggil — itu satu permintaan. Hasilnya dibaca dan diubah jadi jawaban — itu permintaan kedua, dengan riwayat yang sekarang lebih panjang. **Satu panggilan alat berharga dua permintaan**, bukan satu.

Karena itu “berapa alat yang dipanggil” adalah ukuran biaya yang lebih baik daripada “berapa permintaan”, dan kenapa menggabungkan dua alat kecil jadi satu sering lebih hemat daripada mengoptimalkan keduanya.

Waktu tunggu bukan satu angka

Waktu ke token pertama — dipengaruhi panjang masukan. Konteks panjang membuat jeda pertama terasa, dan inilah yang dirasakan pengguna sebagai “lambat”.

Waktu total — dipengaruhi panjang keluaran, sebab token dihasilkan satu per satu. Di agen ini biasanya kecil; yang besar adalah **waktu alat**. Pada agen enam langkah, sebagian besar waktu dinding sering bukan milik model sama sekali — melainkan milik kueri basis data dan panggilan HTTP di dalam alat. Mengukur waktu model saja akan mengoptimalkan bagian yang salah.

Ukur per **giliran**, bukan per tugas: waktu model, waktu alat, dan jumlah giliran. Tiga angka itu memberi tahu di mana waktunya pergi; satu angka total tidak.

Mengalirkan jawaban tidak membuat agen lebih cepat

Untuk agen, pengaruhnya jauh lebih kecil, dan sering **nol** — sebab yang keluar dari model bukan jawaban, melainkan panggilan alat yang harus **lengkap** sebelum bisa dijalankan. Tidak ada yang bisa dikerjakan dari separuh panggilan alat.

Yang benar-benar mengurangi rasa lambat pada agen: **menampilkan alat mana yang sedang berjalan**. Itu sebabnya layar ketiga di demo menyebut nama alatnya, bukan memutar lingkaran.

Tiga cara memaksa bentuk, dengan tiga tingkat jaminan

CARA	JAMINANNYA	KAPAN DIPAKAI
Diminta di prompt	Tidak ada. Sering benar, kadang tidak, dan gagalanya tidak seragam	Prototipe, atau model tanpa dukungan lain
Mode JSON penyedia	JSON-nya sah, isinya belum tentu sesuai skema	Ketika hanya butuh objek yang bisa diurai
Panggilan alat / skema ketat	Nama dan tipe parameter dijamin di sisi penyedia	Untuk agen — ini yang dipakai

Perbedaan baris kedua dan ketiga adalah perbedaan antara *bisa diurai* dan *bisa dipercaya*. Yang pertama menghindarkan galat penguraian; hanya yang kedua yang menghindarkan argumen yang masuk akal secara sintaks dan salah secara isi.

Validasi tetap di sisi Anda, dan gagalnya harus punya jalan

- ① {'h': 'Periksa di batas alat, sebelum efek apa pun', 'p': 'Rentang, keberadaan, kepemilikan. Alat yang dipanggil dengan argumen tidak sah harus **menolak**, bukan menebak maksudnya.'}
- ② {'h': 'Kembalikan galat yang bisa dipakai model', 'p': '"id tidak ditemukan" membuatnya mencoba jalan lain. "Galat 500" membuatnya mengulang hal yang sama.'}
- ③ {'h': 'Hitung berapa kali ini terjadi', 'p': 'Laju penolakan skema adalah ukuran kualitas model yang paling cepat memberi tahu Anda telah memilih model yang salah.'}

Determinisme yang cukup untuk bisa diuji

Suhu nol untuk jalur keputusan

Pemilihan alat dan pengeluaran argumen tidak butuh keragaman. Keragaman di sana namanya ketidakstabilan.

Penyedia luring untuk menguji mesinnya

Gelung, anggaran, penjaga, dan validasi alat tidak butuh model sungguhan untuk diuji — dan uji yang tidak butuh jaringan akan benar-benar dijalankan orang.

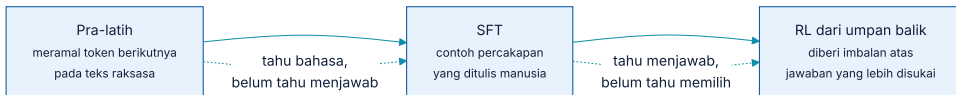
Bahkan pada suhu nol, penyedia **tidak menjanjikan** keluaran yang identik antar waktu. Uji yang mensyaratkan kalimat persis akan rapuh; uji yang mensyaratkan **alat mana yang dipanggil** tidak.

03

Bagaimana ia jadi bisa disuruh

Tiga tahap, dan hanya dua terakhir yang membuatnya berguna sebagai agen.

Tiga tahap, tiga kemampuan yang berbeda



Tiap tahap menambah kemampuan yang tidak dimiliki tahap sebelumnya.

Pra-latih memberinya bahasa dan pengetahuan dunia, dari meramal token berikutnya pada teks dalam jumlah raksasa. Model di tahap ini belum bisa disuruh — ia melanjutkan teks, tidak menjawab pertanyaan.

Tahap yang membuatnya bisa dipakai dalam gelung

Penyetelan terbimbing (SFT)

Dilatih pada contoh percakapan yang ditulis manusia: permintaan, dan jawaban yang pantas. Di sinilah ia belajar bentuk *menjawab*, dan di sini pula ia belajar bentuk **panggilan alat** — sebab contohnya berisi panggilan alat.

Penguatan dari umpan balik

Manusia (atau model penilai) membandingkan dua jawaban; model diberi imbalan atas yang lebih disukai. Di sinilah ia belajar **menuruti batasan**, menolak, dan berhenti men-
garang dengan percaya diri.

Kenapa ini penting bagi pembangun agen: **kepatuhan pada skema alat dan pada instruksi adalah hasil pelatihan, bukan sifat bawaan Transformer**. Dua model dengan arsitektur nyaris sama bisa sangat berbeda di sini, dan itulah yang paling menentukan.

Yang tidak diperbaiki oleh tahap mana pun

- ① {'h': 'Akibat pertama: percaya diri bukan sinyal', 'p': 'Nada yakin dihasilkan oleh pelatihan yang sama, entah isinya benar atau tidak.'}
- ② {'h': 'Akibat kedua: sumber harus dari alat', 'p': 'Angka yang tidak berasal dari hasil alat adalah angka yang tidak punya asal-usul. Itu sebabnya bab 5 memperlakukan alat sebagai sumber bukti, bukan sekadar kemampuan.'}
- ③ {'h': 'Akibat ketiga: penilaian harus melihat jejak', 'p': 'Keluaran yang benar dari alasan yang salah akan berulang. Bab 7.'}

04

Arsitektur, sejauh yang mengubah keputusan

Kedalamannya di Bab 15. Di sini hanya bagian yang terasa di tagihan.

Cache KV: kenapa token pertama lambat dan sisanya cepat

Fase isi — seluruh masukan diproses sekaligus. Sejajar, cepat per token, tapi inilah yang membuat token pertama terasa lama pada konteks panjang.

Fase lanjut — satu token per langkah, memakai cache. Murah dalam komputasi, tetapi cachanya **memakan memori** yang tumbuh bersama panjang konteks.

Yang terasa di produksi: konteks panjang bukan hanya lebih mahal ditagih — ia juga **menurunkan jumlah permintaan bersamaan** yang muat di satu GPU. Itu sebabnya penyedia membedakan harga menurut panjang konteks.

Campuran pakar: kenapa model besar bisa murah

Akibatnya satu angka yang sering dipakai membandingkan model — jumlah parameter — **berhenti berarti apa-apa** sebagai penanda biaya atau kecepatan. Dua model dengan angka yang sama bisa berbeda berkali lipat ongkosnya.

Aturan praktisnya: bandingkan model dengan **biaya per tugas yang selesai**, diukur pada kumpulan uji Anda sendiri. Bukan dengan jumlah parameter, dan bukan dengan harga per juta token.

Ke mana mencari kedalamannya

KALAU INGIN TAHU	LIHAT	YANG ADA DI SANA
Cara perhatian diri menghitung	Bab 15	Q·K·V pada kalimat nyata sampai persentase softmax, dan satu slide yang membaca angkanya dengan jujur
Kenapa perlu proyeksi Q dan K	Bab 15	Jejak run : tanpa proyeksi, sebuah kata menghabiskan 60% perhatiannya untuk dirinya sendiri
Penyandian posisi	Bab 15	Gambar yang menghitung sendiri
Menghasilkan teks, pengambilan sampel	Bab 16	Suhu, top-p, dan gelung pengambilan sampel

Dek ini sengaja berhenti di sini. Menuliskan ulang isi Bab 15 dengan kata yang berbeda akan menambah slide tanpa menambah apa pun yang bisa dipakai.

05

Memilih model untuk sebuah agen

Papan skor menjawab pertanyaan yang berbeda dari pertanyaan Anda.

Tiga sifat yang menentukan, dan hanya satu yang ada di papan skor

Kepatuhan pada skema

Seberapa sering ia mengeluarkan panggilan alat yang **sah** dan argumen yang bertipe benar. Kegagalan di sini terlihat seperti agen yang bodoh, padahal soal format.

Kepatuhan pada instruksi panjang

Batasan di token ke-3 000 masih diikuti pada giliran ke-15? Inilah yang paling cepat rusak saat konteks memanjang.

Kemauan berhenti

Seberapa sering ia mengatakan *tidak bisa* alih-alih menebak. Model yang tidak pernah menyerah menghasilkan agen yang berputar.

Papan skor umum mengukur pengetahuan dan penalaran — berguna, tetapi bukan tiga hal di atas. Ketiganya harus **diukur pada kumpulan uji Anda sendiri**, dan itu satu sore kerja, bukan satu proyek.

Urutan yang menghemat paling banyak waktu

- 1 {'h': 'Mulai dari model paling mampu yang ada', 'p': 'Sampai agennya **benar**. Mengoptimalkan agen yang belum benar hanya membuat kesalahannya lebih murah.'}
- 2 {'h': 'Baru turunkan, satu tingkat, dengan kumpulan uji di tangan', 'p': 'Kalau angkanya bertahan, simpan penghematannya. Kalau tidak, Anda baru saja mengetahui harga sebenarnya dari model yang lebih murah.'}
- 3 {'h': 'Ukur biaya per tugas selesai, bukan per permintaan', 'p': 'Model murah yang perlu dua kali lebih banyak giliran tidak menghemat apa pun.'}
- 4 {'h': 'Pertimbangkan model lokal ketika datanya yang menentukan', 'p': 'Bukan karena lebih murah — sering tidak. Karena ada data yang tidak boleh keluar, dan itu keputusan kepatuhan, bukan teknis.'}

`ai-agentic-demo` menjalankan kelimanya lewat satu antarmuka: `echo` (luring), `ollama` (mesin sendiri), `gemini`, `anthropic`, `openai` — jadi menukar model adalah satu argumen, bukan satu penulisan ulang.

Bahasa Indonesia lebih mahal per kalimat, dan itu bukan dugaan

Akibatnya langsung dan berlipat: kalimat yang sama artinya menghabiskan lebih banyak token, jadi lebih mahal, lebih cepat memenuhi jendela konteks, dan lebih lambat dihasilkan.

Jangan menaksir — **ukur pada teks Anda sendiri**. Ambil seratus kalimat nyata dari domain Anda, hitung tokennya dengan pemanggil model yang dipakai, dan pakai angka itu untuk perencanaan biaya. Taksiran 4 karakter per token adalah taksiran untuk bahasa Inggris.

Jendela besar tidak berarti seluruhnya terpakai dengan baik

Akibat untuk penyusunan konteks

- ♦ Taruh perintah dan batasan di **awal**
- ♦ Taruh permintaan giliran ini di **akhir**
- ♦ Jangan menaruh aturan penting di tengah tumpukan hasil alat

Akibat untuk arsitektur

- ♦ Konteks besar bukan pengganti pengambilan yang tepat
- ♦ Memasukkan seluruh dokumen karena “muat” menu-runkan ketepatan, bukan menaikkannya
- ♦ Bab 4 membahas apa yang disimpan dan apa yang dicari lagi

Uji ini murah dan jarang dilakukan: **sisipkan satu fakta di tengah konteks panjang dan tanyakan** — pada panjang yang benar-benar Anda pakai, bukan pada panjang maksimum di brosur.

Model diperbarui, dan agen Anda berubah tanpa Anda menyentuhnya

- ① {'h': 'Sematkan versinya di produksi', 'p': 'Kalau penyediaanya menyediakan pengenalan versi, pakai. Kalau tidak, catat tanggal dan jalankan kumpulan uji lebih sering.'}
- ② {'h': 'Catat versi model pada tiap jejak', 'p': 'Tanpa itu, "sejak kapan ini mulai salah" tidak bisa dijawab. Sama seperti versi model kredit dicatat pada tiap rekomendasi di demo.'}
- ③ {'h': 'Jalankan kumpulan uji sebelum berpindah versi', 'p': 'Perpindahan versi adalah perubahan perangkat lunak, dan pantas diperlakukan seperti perubahan perangkat lunak.'}

Ini kelas kegagalan yang tidak muncul di pengembangan dan hanya muncul di produksi — dan gejalanya **selalu** terlihat seperti "tiba-tiba agennya jadi bodoh".

Batas laju: yang membatasi bukan kecepatan, melainkan token per menit

Kenapa ini mengejutkan orang

Sepuluh pengguna bersamaan terdengar kecil. Sepuluh pengguna $\times$ enam giliran $\times$ konteks yang menumpuk bisa berarti ratusan ribu token per menit.

Yang harus ada sebelum dipakai banyak orang

- ♦ Antrean, bukan pengulangan langsung
- ♦ Mundur bertahap ketika ditolak
- ♦ Batas jumlah proses agen yang berjalan bersamaan

Dan satu akibat desain: **memperpendek konteks menaikkan kapasitas**, bukan hanya menurunkan biaya. Keduanya obat yang sama.

Model di mesin sendiri: kapan masuk akal

Ketika datanya yang menentukan

Ada data yang tidak boleh keluar dari perimeter. Ini keputusan **kepatuhan**, dan ia mengalahkan pertimbangan biaya maupun kualitas.

Bukan karena lebih murah

Sering justru tidak, kalau pemakaiannya tidak rata: GPU mengganggu tetap dibayar, sedangkan API yang tidak dipanggil tidak.

Yang berubah secara teknis: model terbuka umumnya lebih lemah pada **kepatuhan skema alat** dan lebih mudah berhenti terlalu awal. Itu bukan alasan menolaknya — itu alasan mengukurnya pada kumpulan uji Anda sebelum memutuskan.

Dijalankan sungguhan di demo dengan `qwen3:8b`: gelungnya benar, tetapi **berhenti satu langkah lebih awal** tanpa memanggil alat pengiriman rekomendasi. Kelemahan yang terukur, bukan yang ditebak.

Enam angka yang pantas dicatat pada tiap proses

ANGKA	YANG DIBERITAHUKANNYA	GEJALA KALAU MEMBURUK
Token masukan per tugas	Biaya sebenarnya	Naik pelan tanpa perubahan kode → riwayat tidak dipangkas
<code>cache_read</code> bukan nol	Singgahan menyala	Jadi nol → ada yang menaruh sesuatu yang berubah di awalan
Giliran per tugas	Panjang penalaran	Naik → alat berubah, atau deskripsinya jadi kabur
Laju penolakan skema	Kecocokan model	Naik sesudah ganti model → modelnya, bukan agennya
Waktu alat lawan waktu model	Ke mana waktu pergi	Waktu alat mendominasi → optimasi model tidak akan terasa
Laju eskalasi ke manusia	Kejujuran sistem	Turun ke nol → curigai, jangan rayakan

Tidak satu pun dari enam ini menimbulkan **galat**. Semuanya bergeser diam-diam, dan hanya terlihat kalau dicatat sejak awal.

Yang dibawa pulang dari bab ini

- ① {'h': 'Token adalah satuan segalanya', 'p': 'Tagihan, waktu tunggu, dan batas konteks — ketiganya diukur dengan satuan yang sama, dan agen adalah beban **masuk**an.'}
- ② {'h': 'Percakapan sepuluh giliran menagih enam kali isinya', 'p': 'Sebab tiap giliran membayar ulang semua giliran sebelumnya. Mengurangi langkah mengalahkan memangkas kata.'}
- ③ {'h': 'Model tidak mengingat; daftar pesan Anda yang mengingat', 'p': 'Itu kabar baik — keadaan yang Anda pegang bisa diperiksa dan dipotong dengan sengaja.'}
- ④ {'h': 'Panggilan alat adalah niat, bukan eksekusi', 'p': 'Semua batas izin hidup di celah antara model menuliskannya dan kode Anda menjalankannya.'}
- ⑤ {'h': 'Pilih model dengan kumpulan uji Anda, bukan papan skor', 'p': 'Kepatuhan skema, kemauan berhenti, dan kepatuhan instruksi panjang tidak ada di sana.'}